

Vayu flight-log analysis — 20260617-124210

bench-rig capture, decoded via NavLink v2

Contents

Log archive — 2026-06-17 12:42:10	3
Contents	3
Provenance & integrity	3
Container header (VREC, software/src/replay/RecordFormat.h)	3
Message inventory	4
Raw findings log (everything we dug out)	4
Log analysis — export-20260617-124210.bin	6
TL;DR	6
Session overview	6
Flight timeline	7
Controller behaviour (ARMED windows only)	8
RC link & failsafe (works correctly)	9
Health, timing, link	9
Data-quality issues found (firmware-side telemetry bugs)	10
Recommended follow-ups	10
Control-loop analysis — export-20260617-124210.bin	11
Architecture (from firmware)	11
Loop timing — healthy	11
The “level command but tilted” finding	11
The oscillation (why throttle had to be pulled)	13
Yaw — disturbance rejection, saturation, and the hand-clamp episodes	14
Takeaways	14
Motor analysis — export-20260617-124210.bin	16
Mixer & geometry (from firmware)	16
Output levels — no saturation, low throttle	16
Balance under a level, throttle-only command	16
What this log can and can’t tell us about the motors	17
Recommendations	17
Kernel / RTOS analysis — export-20260617-124210.bin	18
Headline	18
Dataflow: why telemetry drops but control doesn’t	18
CPU & scheduler	19
Estimator timing (EstPerf)	19
Memory	19
IPC	19
FIFOs — the one real signal	19
Takeaways	20
Sensor & fusion report — export-20260617-124210.bin	21
Verdict at a glance	21
Gyroscope — good	21

Accelerometer — ~8 % scale error	21
Magnetometer — uncalibrated (do not use as-is)	22
Temperature — good	22
Sensor fusion (attitude estimation)	22
Reporting fidelity	24
Recommendations (sensor priority order)	24
Recommendations & next-run plan — export-20260617-124210.bin	26
1. Fixes before the next rig trial (priority order)	26
2. What to capture in the next rig run (test protocol)	26
3. What specifically to capture (states & methods)	27
4. Rig acceptance criteria (gate before <i>any</i> flight thinking)	27

Log archive — 2026-06-17 12:42:10

Self-contained analysis archive for one real-hardware telemetry capture. The raw `.bin` is kept here so every number can be re-derived later.

Contents

file	what
20260617-124210-analysis.pdf	all docs below combined into one printable PDF (with plots & diagrams)
export-20260617-124210.bin	raw capture (source of truth)
session-analysis.md	full session overview (link, flight timeline, controller, health)
control-loop-analysis.md	cascade controller: the attitude-loop oscillation/instability , resting tilt, yaw spins
motor-analysis.md	quad-X mixer, motor balance, saturation
kernel-analysis.md	vaios RTOS: CPU, stacks, heap, IPC, FIFO drops
sensor-analysis.md	sensor calibration + fusion (EKF) accuracy & reporting
recommendations.md	fixes before next trial + what to capture next (rig-only)
plots/	figures embedded in the docs above (committed; regenerate with <code>make_plots.py</code>)
csv/	per-message-type CSVs for plotting (git-ignored, regenerable)

Decoder lives one level up: `../parse_log.py`. Reproduce everything from repo root:

```
python3 docs/log-analysis/parse_log.py docs/log-analysis/20260617-124210/export-20260617-124210.bin
python3 docs/log-analysis/parse_log.py docs/log-analysis/20260617-124210/export-20260617-124210.bin --csv
  ↪ docs/log-analysis/20260617-124210/csv
python3 docs/log-analysis/make_plots.py docs/log-analysis/20260617-124210 # regenerate plots/
python3 docs/log-analysis/build_pdf.py docs/log-analysis/20260617-124210 # rebuild the combined PDF
```

Provenance & integrity

Original path	/home/ragnar/vayu-logs/export-20260617-124210.bin
Archived	2026-06-18
Size	2,032,071 bytes
SHA-256	8b4f82ccb50a7166c23fd9149f51069063fd522de7b40df86898ad5aed039e

Container header (VREC, `software/src/replay/RecordFormat.h`)

field	value
magic	0x56524543 ("VREC")
format version	1
protocol version	1
start wall clock	1781680333706 ms → 2026-06-17 07:12:13 UTC
records (byte chunks)	38,292
decoded frames	38,292
CRC errors	0
unknown msgids	none
time span	375.2 s (6.25 min), <code>t_us</code> 31810007043 ... 32185159631

field	value
trailing bytes	0

Message inventory

msgid	message	count	rate (Hz)
1025	ImuCompressed	8,764	23.4
1030	ControlTrace	7,450	19.9
1029	MotorTelemetry	6,562	17.5
1026	AttitudeEuler	3,520	9.4
1028	RcChannels	3,458	9.2
1035	PerfTask	3,173	8.5
0	Heartbeat	1,078	2.9
1036	PerfFifo	1,062	2.8
3	FlightMode	1,015	2.7
2	SystemHealth	969	2.6
1024	ImuRaw	508	1.4
1034	PerfGlobal	372	1.0
1033	EstPerf	284	0.8
10	TimeSync	77	0.2

Absent: Statustext, CommandAck, Ping, Capabilities, Param*, CalibrationStatus, all Cmd*.

Raw findings log (everything we dug out)

Quick reference; see the per-domain analysis docs for context and method. **Rig context (operator):** captured on a bench rig (a free pivot) — never `IN_AIR`. At rest the frame hangs to one side; on throttle the roll/pitch loop tries to level, then **breaks into a growing oscillation, so throttle had to be pulled**. Yaw was hand-spun/clamped.

Decode/transport - 0 CRC errors / 0 unknown msgids over 38,292 frames — wire stack is clean on HW.
- Avg downlink ~4.2 kB/s (33.5 kbps). 9 record gaps > 200 ms (max 312 ms). - SystemHealth.tx_overflow 612 → 2680 (~2,068 telemetry TX-buffer drops): link saturated.

Flight state - Armed 4× (19.6 s, 98.7 s, 44.0 s, 50.7 s) — **never** `IN_AIR`. Bench rig. - Mode ANGLE throughout except ACRO 185–219 s. All mode changes source=RC. - Every RC dropout (chan 0/1 → 62954 = −2582 as int16) → FAILSAFE (615/620 spike frames). Safety path correct.

Control loop (detail) - **INSTABILITY: roll/pitch loop diverges into a ~0.3 Hz limit cycle** (roll to ±50–67°) whenever throttle is held up; operator pulled throttle to stop it in all 3 long windows. Output anti-phase with angle (corr −0.82) ⇒ closed-loop, not passive swing. **Fix before flight.** - Likely cause: rate Kp roll/pitch = 0.0005 with **Kd = 0** and integral-dominant Ki = 0.01 → underdamped; add Kd, raise Kp, lower Ki/i_max. - Resting pose (rig free pivot): frame hangs at **pitch +25°, roll −10°** — real (fusion matches gravity), not a sensor lie. - Yaw Kp 0.018 = **36× roll/pitch**; yaw_rate_sp=0 always; frame disturbed ±80 °/s; yaw_out saturates 0.5 % of frames; 343–351 s alternating ±65–80 °/s spins = hand-spin/clamp (possible separate yaw limit-cycle). - Loop timing 250 Hz outer / 1 kHz inner, **zero jitter** → instability is tuning, not scheduling.

Motors (detail) - Quad-X mixer signs verified vs firmware. M1 FR/CW, M2 RR/CCW, M3 RL/CW, M4 FL/CCW; cmd[] 0..1. - Peak 0.69, **never saturated** (0 frames > 0.95). Throttle-only imbalance (M1 −19 %, M2 +14 %) is the controller's corrections mixed in, not a raw actuator fault — can't assess motor health from a rig.

Kernel / vaio (detail) - CPU ~63 % (steady, via cycle deltas), estimator EKF 250 Hz using 22 % of its 4 ms budget. - Heap 0 OOM (37 % peak), IPC 0 timeouts, stacks in margin (worst task_id 4 = 74 %). - **FIFO drops are telemetry-only**: control FIFOs 0 drops; telemetry FIFOs pinned 100 %, drop counter saturated at u16 65535. Control path never starved.

Sensors & fusion ([detail](#)) - Gyro: bias < 0.1 °/s, noise σ 0.3–0.5 °/s — OK good. - Accel: +7.8 % scale error at rest (motors off); not vibration — WARN needs cal. - Mag: $|mag|$ 22–92 μ T (140 % of mean), hard-iron offsets to 25 μ T — FAIL uncalibrated. - Temp: 35.6–37.1 °C, smooth — OK good. - **Fusion (EKF)**: roll/pitch accurate (fused tilt 33.7° vs accel 37.5°, $\Delta -3.8^\circ$) — tilt is real OK. **Yaw untrustworthy** (pinned by uncalibrated mag) FAIL. - Units: gyro °/s, accel m/s², mag μ T, temp °C.

Telemetry data-quality bugs (firmware-side) 1. AttitudeEuler.{rollspeed,pitchspeed,yawspeed} always 0 (body rates not populated). 2. SystemHealth.cpu_load always 0. 3. ImuRaw.sample_time_us always 0. 4. ImuCompressed.ref_seq always 0 — broken keyframe linkage; compressed stream not reconstructable. 5. Unit mismatch: ControlTrace angles in degrees vs AttitudeEuler in radians (verified 1.00 ratio).

Calibration evidence - 0 CalibrationStatus messages; nav_state never CALIBRATING. No calibration captured this session.

Log analysis — export-20260617-124210.bin

First analysis of a **real-hardware** telemetry capture. Decoded with `../parse_log.py` via the generated NavLink v2 codec — every figure below is reproducible from the archived `.bin` in this folder.

from repo root

`python3 docs/log-analysis/parse_log.py docs/log-analysis/20260617-124210/export-20260617-124210.bin`

TL;DR

- **Healthy link decode.** 38,292 frames, **0 CRC errors** across 6.25 min — the wire format, framing and CRC are solid end-to-end on hardware.
- **The aircraft never flew.** `nav_state` reached ARMED four times but **never IN_AIR**; throttle stayed ≤ 0.3 and motors ≤ 0.69 . This is a **bench-rig** session (a free pivot), not flight.
- **The roll/pitch attitude loop is unstable on the rig.** With throttle held up it diverges into a **~0.3 Hz limit cycle** (roll to $\pm 50\text{--}67^\circ$); the operator had to pull throttle to stop it, three times. Controller output is anti-phase with angle (corr -0.82) — a real closed-loop instability, almost certainly the rate-loop tuning (**Kd = 0**, tiny Kp, integral-dominant). See [control-loop-analysis.md](#). **Fix before flight.**
- **RC-loss → FAILSAFE works.** Every RC dropout (chan 0/1 → 62954) lined up with a FAILSAFE transition (615/620 spike frames). Safety path behaves correctly.
- **Four telemetry data-quality bugs** worth fixing (details below): three always-zero fields and one unit mismatch.
- **Telemetry link is saturated.** `tx_overflow` grew $\sim 612 \rightarrow 2680$ ($\sim 2,068$ dropped TX buffers) at ~ 33.5 kbps average; 9 link stalls > 200 ms.

Session overview

Container	VREC format v1, protocol v1
Start wall clock	1781680333706 ms → 2026-06-17 07:12:13 UTC
Duration	375.2 s (6.25 min)
Records (byte chunks)	38,292
Decoded frames	38,292
CRC errors	0
Unknown msgids	none
Avg telemetry rate	1,572,547 payload bytes → ~ 4.2 kB/s (33.5 kbps)

Message mix

message	count	rate (Hz)	notes
ImuCompressed	8,764	23.4	delta IMU
ControlTrace	7,450	19.9	PID/loop trace
MotorTelemetry	6,562	17.5	4 motors used
AttitudeEuler	3,520	9.4	body rates all 0 WARN
RcChannels	3,458	9.2	14 channels
PerfTask	3,173	8.5	vaivos scheduler
Heartbeat	1,078	2.9	nav state
PerfFifo	1,062	2.8	
FlightMode	1,015	2.7	
SystemHealth	969	2.6	<code>cpu_load</code> all 0 WARN
ImuRaw	508	1.4	<code>sample_time_us</code> all 0 WARN
PerfGlobal	372	1.0	
EstPerf	284	0.8	estimator timing
TimeSync	77	0.2	

No Statustext, CommandAck, or Param* traffic in this session.

Flight timeline

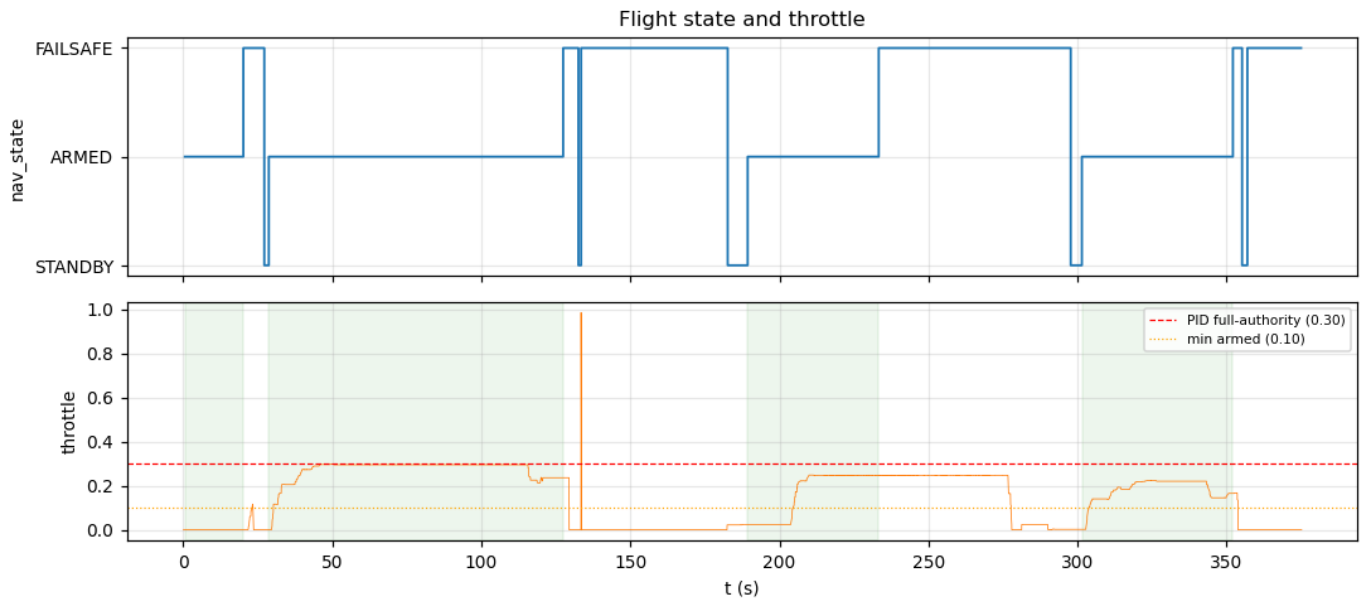
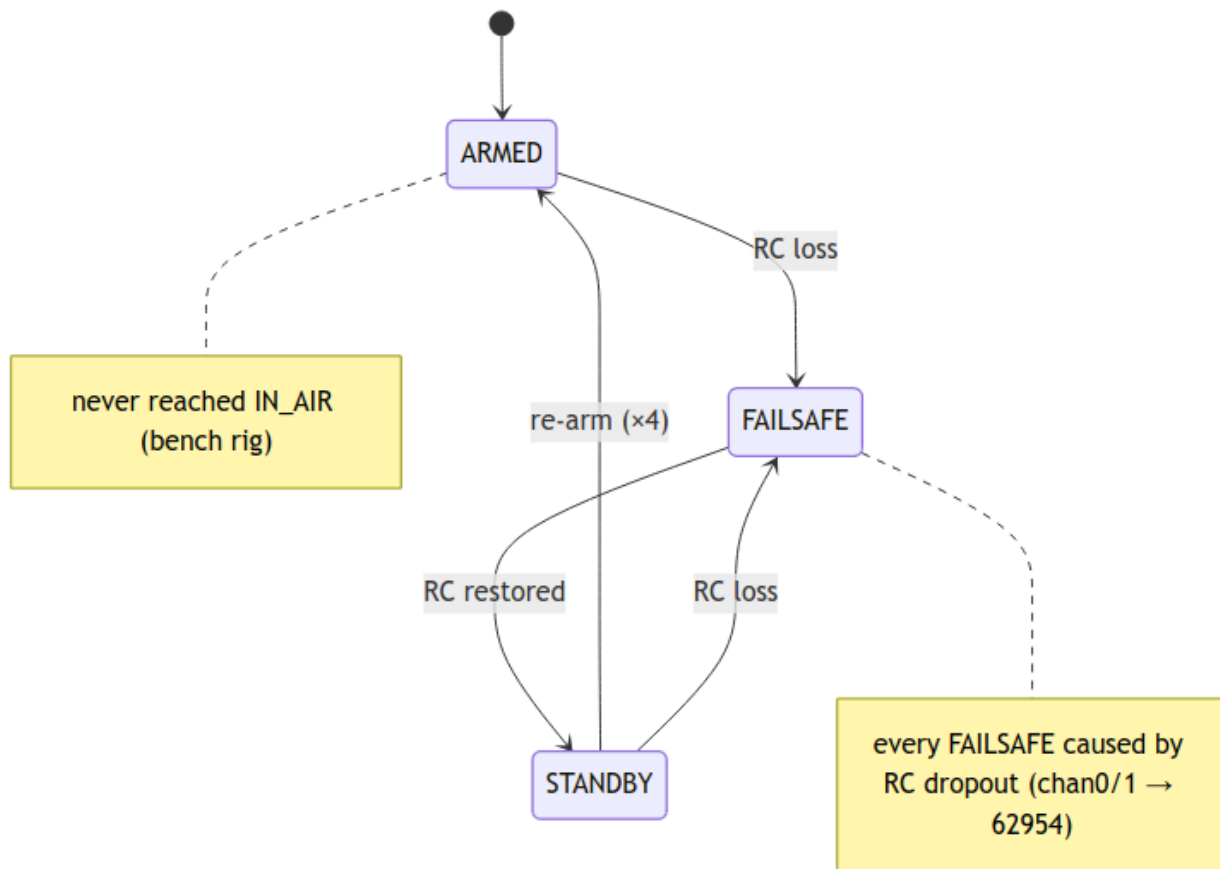


Figure 1: session overview

Observed `nav_state` machine (transitions seen in this log; `IN_AIR`, `PREARM`, `TERMINATED`, `CALIBRATING` never reached):



nav_state (from Heartbeat) and flight mode (FlightMode), times relative to first frame:

```

t= 0.5s  ARMED                      mode ANGLE (RC)
t= 20.1s  FAISAFE      ← RC loss
t= 27.1s  STANDBY
t= 28.7s  ARMED                      (longest window, 98.7 s)
t=127.4s  FAISAFE      ← RC loss
t=132.5s  STANDBY → FAISAFE (133.4s, RC still out until 182.6s)
t=182.6s  STANDBY
t=185.1s  ————— mode ACRO (RC)
t=189.3s  ARMED
t=218.7s  ————— mode ANGLE (RC)
t=233.3s  FAISAFE      ← RC loss (until 297.7s)
t=301.4s  ARMED
t=352.1s  FAISAFE → STANDBY → FAISAFE (end)
  
```

Four **ARMED** windows totalling ~213 s: 19.6 s, 98.7 s, 44.0 s, 50.7 s. The system **never entered IN_AIR**. A brief switch to **ACRO** mode at 185–219 s is the only excursion from ANGLE. All mode changes were source=RC.

The fused attitude over the whole session — note the sustained positive pitch (the frame rested nose-up on the rig) and the full-range yaw wander:

Controller behaviour (ARMED windows only)

Restricting ControlTrace to the four ARMED windows (4,225 frames). **Angles are in degrees, rates in deg/s** (see unit note below):

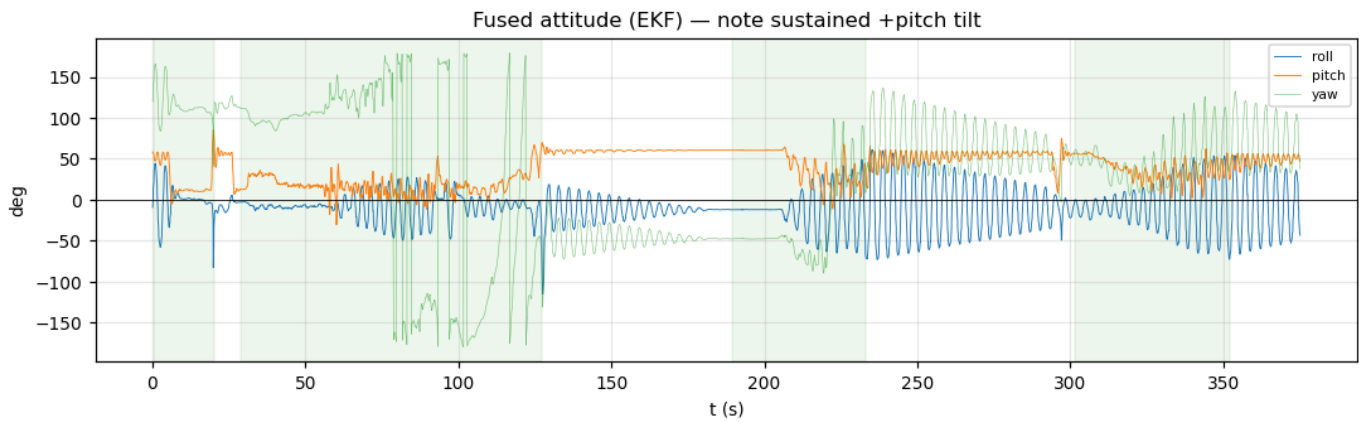


Figure 2: fused attitude

signal	min	max	mean	rms
roll angle error (°)	-53.5	81.2	9.0	23.3
pitch angle error (°)	-85.3	30.3	-27.4	33.1
throttle out	0.0	0.3	0.2	0.2

Motor commands during ARMED (0–1 scale): each of motors 0–3 mean ≈ 0.2 , max ≈ 0.6 – 0.69 , **never saturated** (0 frames > 0.95). The large attitude errors here are mostly the **closed-loop oscillation**, not hand motion: with throttle held up the roll/pitch loop diverges into a ~ 0.3 Hz limit cycle and the operator pulls throttle to stop it (see [control-loop-analysis.md](#) for the per-episode breakdown and root-cause). Loop timing is rock-solid: $\text{outer_dt} = 4.000$ ms (250 Hz), $\text{inner_dt} = 1.000$ ms (1 kHz), zero jitter — so the instability is a **tuning** problem, not a scheduling one.

Deep-dives split out by subsystem: [control-loop-analysis.md](#) (the **attitude-loop oscillation**, resting tilt, yaw spins), [motor-analysis.md](#) (mixer & balance), [kernel-analysis.md](#) (RTOS health), [sensor-analysis.md](#) (sensors + EKF fusion).

RC link & failsafe (works correctly)

RcChannels: count=14, rssi=0 (RSSI not wired). Channels 2–5 look like normal PWM (1000–2000 μ s); chan 3 constant 1500.

Channels **0 and 1 jump to 62954 / 62940** in 620 frames — **615 of them while** `nav_state == FAILSAFE`. 62954 interpreted as `int16` is **-2582**, i.e. these two channels carry signed stick values in a `u16`, and the receiver emits an out-of-range sentinel on RC loss. The FC correctly detects this and transitions to FAILSAFE every time. **This is correct, expected safety behaviour**, not a bug — but a consumer plotting `chan[0..1]` raw must treat them as signed and mask the failsafe sentinel.

Health, timing, link

- **SystemHealth.tx_overflow 612 → 2680**. $\sim 2,068$ telemetry TX-buffer overflows during the session: the downlink can't keep up with the telemetry rate. `imu_drop = 0`, `log_wrap = 0` (good).
- **9 record gaps > 200 ms** (max 312 ms) — link stalls, consistent with the TX overflow pressure. Average throughput 33.5 kbps.
- **EstPerf**: estimator runs at a steady **250 Hz**, mean 709 μ s, peak 891 μ s — comfortable headroom inside the 4 ms budget ($\sim 22\%$ peak).
- **IMU**: temp 35.6–37.1 °C (sane). `acc` $\approx$ m/s² (vector $\sim$ g). `gyr` is in **deg/s** (abs max ~ 97 , matching `ControlTrace.roll_rate_curr` max ~ 102), not rad/s as the field comment guesses.

Data-quality issues found (firmware-side telemetry bugs)

These are populate/units bugs in what the FC *sends*, not decode errors:

1. **AttitudeEuler.{rollspeed,pitchspeed,yawspeed} are always 0** (0/3520 nonzero). Body rates are not being filled in. This matches the deferred “attitude under-read” issue (docs/... attitude diagnosis). The data exists — `ControlTrace.*_rate_curr` and `ImuRaw.gyr` carry real body rates — so the ATTITUDE_EULER producer just isn’t copying them.
2. **SystemHealth.cpu_load is always 0**. CPU load isn’t populated here; real timing lives in `Est-Perf/PerfGlobal`. Either wire it up or stop advertising it.
3. **ImuRaw.sample_time_us is always 0**. Per-sample timestamp not set, so IMU frames can’t be time-aligned from their own payload (we fall back to the container `t_us`).
4. **Angle-unit inconsistency between messages**. `ControlTrace` angles are in **degrees** while `AttitudeEuler` angles are in **radians** (verified: at matched timestamps $\text{AttitudeEuler.roll} \times 180/\pi \equiv \text{ControlTrace.roll_angle_curr}$, ratio 1.00). Pick one convention (the spec/NED norm is radians) or document the per-field units explicitly, or a consumer will mis-scale by 57×.

Recommended follow-ups

- **Fix the attitude-loop instability first** (see [control-loop-analysis.md](#)): re-tune the roll/pitch rate loop (add `Kd`, raise `Kp` off the integrator, lower `Ki/i_max`) and confirm on the rig that throttle-up no longer oscillates **before** any flight attempt.
- Fix the three always-zero fields (#1-#3) — #1 (body rates) is the most impactful for any attitude consumer/replay.
- Resolve the °/rad inconsistency (#4) in `dialect.json` field docs and the firmware producers.
- Investigate the telemetry **TX saturation** (2k overflows): either raise the downlink bitrate, decimate the high-rate streams (`ImuCompressed` 23 Hz + `ControlTrace` 20 Hz + `MotorTelemetry` 17 Hz dominate), or apply the proposed stream-rate control packet (0x2007) to throttle on-air rates.
- Next capture is **rig-only**: after re-tuning, verify a stable throttle-up and disturbance recovery on the rig (see [recommendations.md](#)). No free flight until the rig is boring.

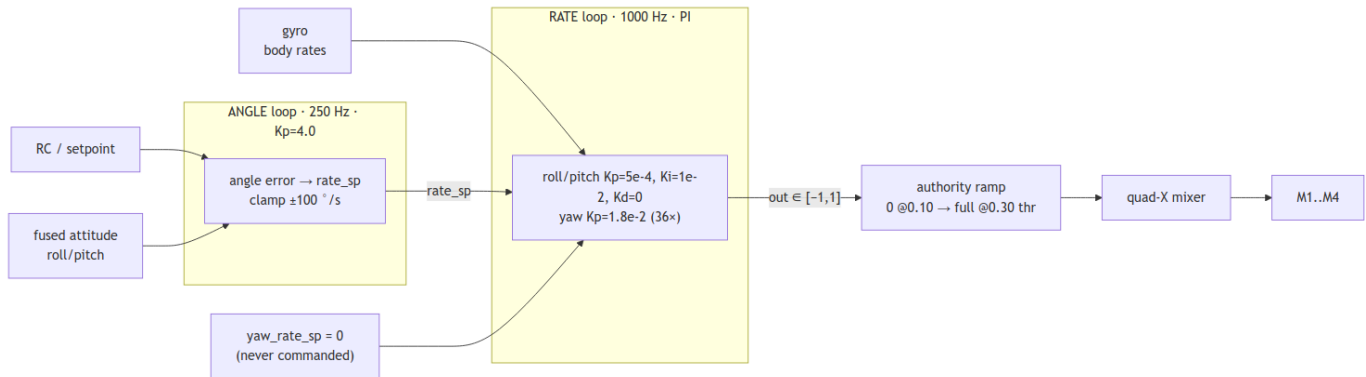
Control-loop analysis — export-20260617-124210.bin

Cascade controller behaviour from ControlTrace (msgid 1030, 7,450 frames), restricted to the four **ARMED** windows. **Context (from the operator):** this was a **bench-rig** run — the airframe never flew (IN_AIR never reached). The rig is a *free pivot*: at rest the frame **hangs/swings to one side** (that is the resting pose), and on throttle the controller **tries to level it, partly succeeds, then breaks into a growing oscillation that forced the operator to pull throttle**. Yaw was also hand-spun/clamped. The data backs every part of this — see the [oscillation section](#). Firmware refs ground every interpretation. Companion to [session-analysis.md](#) and [motor-analysis.md](#).

Units (firmware control_telemetry_t, include/variables.h:243): **angles in degrees, rates in deg/s, *_out** $\in [-1, +1]$ feeding the mixer.

Architecture (from firmware)

Cascade, two rates:



- Outer OUTER_LOOP = 250 Hz (outer_dt = 4.000 ms, **zero jitter** in log).
- Inner INNER_LOOP = 1000 Hz (inner_dt = 1.000 ms, **zero jitter**).
- Default gains (include/variables.h):
 - **Angle (outer):** roll/pitch Kp = 4.0, out clamp ± 100 °/s. Yaw is rate-controlled (no outer angle loop in flight).
 - **Rate (inner):** roll/pitch **Kp = 0.0005**, Ki = 0.01, Kd = 0, i_max 0.2. **Yaw Kp = 0.018**, Ki = 0.008 — i.e. **yaw P-gain is 36× roll/pitch**.
 - **PID-authority ramp:** outputs scale from 0 at MIN_ARMED_THROTTLE (0.10) to full at PID_FULL_AUTHORITY_THROTTLE (0.30) to stop ground feedback.

Loop timing — healthy

outer_dt = 4.000 ms and inner_dt = 1.000 ms with **zero variance** across all 7,450 frames. The 250 Hz / 1 kHz cascade is keeping schedule exactly (corroborated by the kernel EstPerf 250 Hz and PerfTask timing — see [kernel-analysis.md](#)). No timing pathology.

The “level command but tilted” finding

With a **level command** ($|\text{roll_sp}|, |\text{pitch_sp}| < 2^\circ$) and throttle up, the airframe sat at:

	mean	std
roll_angle_curr	-9.7°	22.6°
pitch_angle_curr	+25.1°	16.3°

So pitch held **~+25° nose-up** against a 0° command.

Tracing the cascade in these frames shows the controller is doing the *right thing in the right direction* — it just has almost no authority:

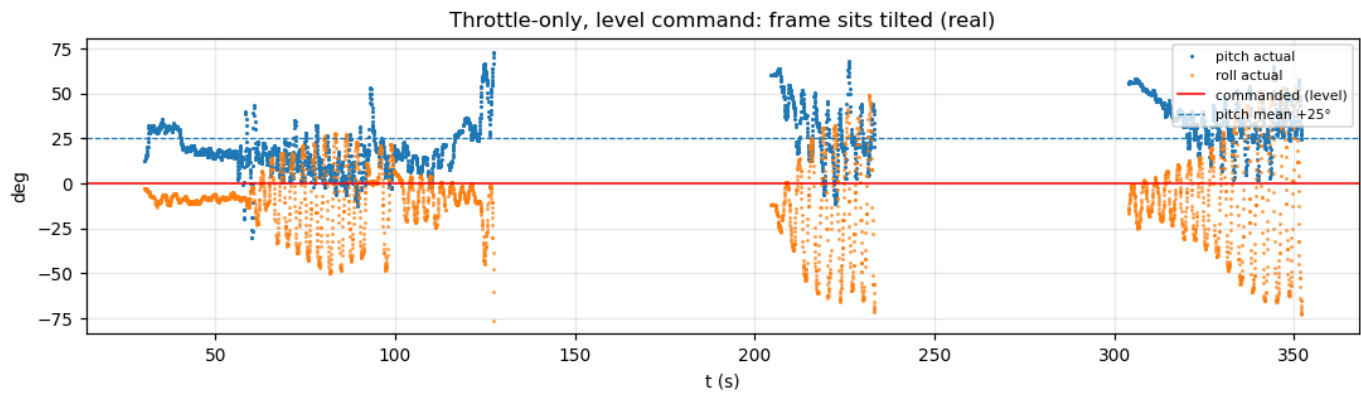


Figure 3: throttle-only tilt

pitch angle error -25 deg --[Kp=4]--> pitch_rate_sp -66 deg/s (outer loop OK, commands nose-down)
pitch rate error -66 deg/s --[Kp=5e-4]--> pitch_out -0.023 (inner loop output ~zero)

The inner-loop output is tiny **by design**: empirical output/error slope is **0.00033** for pitch and **0.00034** for roll — matching the firmware $K_p = 0.0005$ (the rest leans on the slow $K_i = 0.01$ integral, clamped at 0.2). With $K_d = 0$ there is no rate damping either. Net: roll/pitch have very low proportional authority.

The authority asymmetry is stark when you plot controller output against rate error per axis — roll/pitch barely leave zero while yaw spans the full ± 1 :

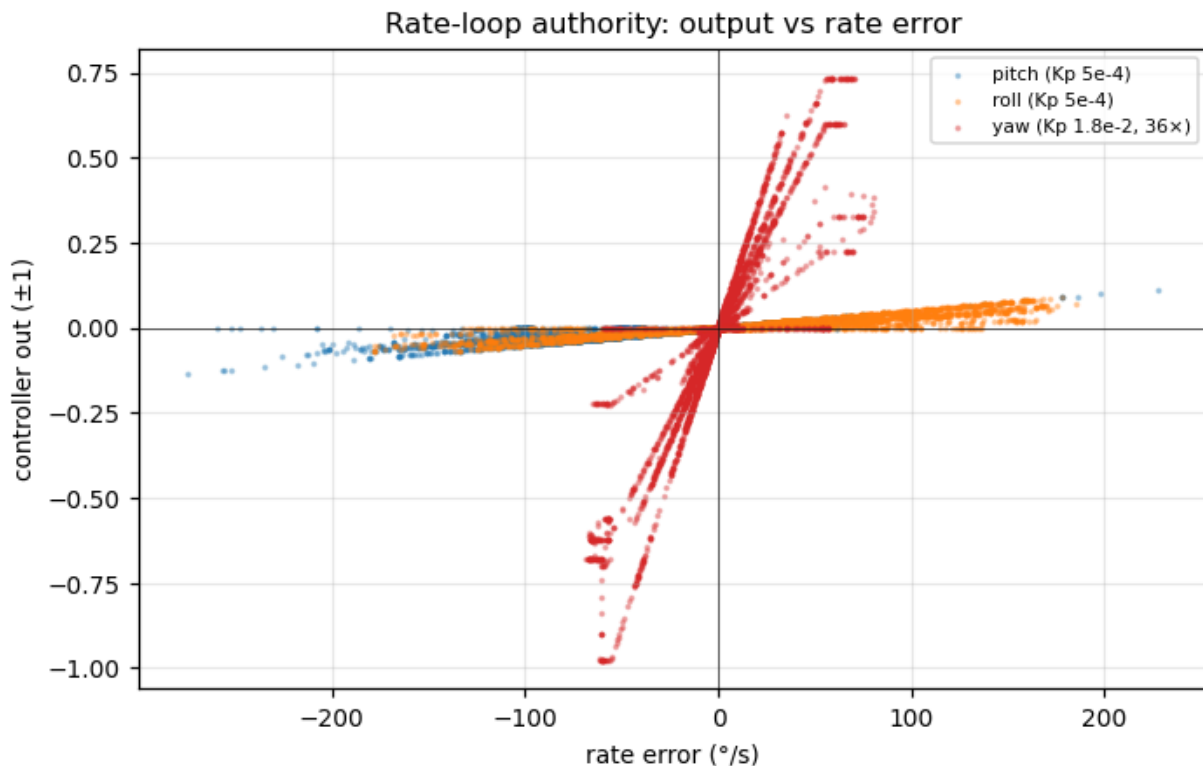


Figure 4: rate-loop authority

This is compounded by the **authority ramp**: armed throttle averaged **0.25** and was **below the 0.30 full-authority threshold 100 % of the time**, so roll/pitch output was *additionally* attenuated ($\approx 0.5\times$) the entire session.

Conclusion. The mean $+25^\circ$ pitch / -10° roll is partly the rig's natural **resting hang** (the operator

notes the frame settles to one side at rest) and partly the swing of the oscillation below. The tilt is *real* (fused attitude matches gravity — see [sensor-analysis.md](#) §Fusion). The earlier reading that “the controller can’t move a constrained frame” was **wrong**: the rig is a free pivot, the controller *does* move it, and as authority comes up with throttle it drives the frame into a **growing oscillation** rather than settling level. That instability — not a lack of authority — is the real story.

The oscillation (why throttle had to be pulled)

This is the key finding. In every armed window, once throttle is held up the roll/pitch attitude **builds into a sustained, growing oscillation**, and the only thing in the log that stops it is the operator **cutting throttle**.

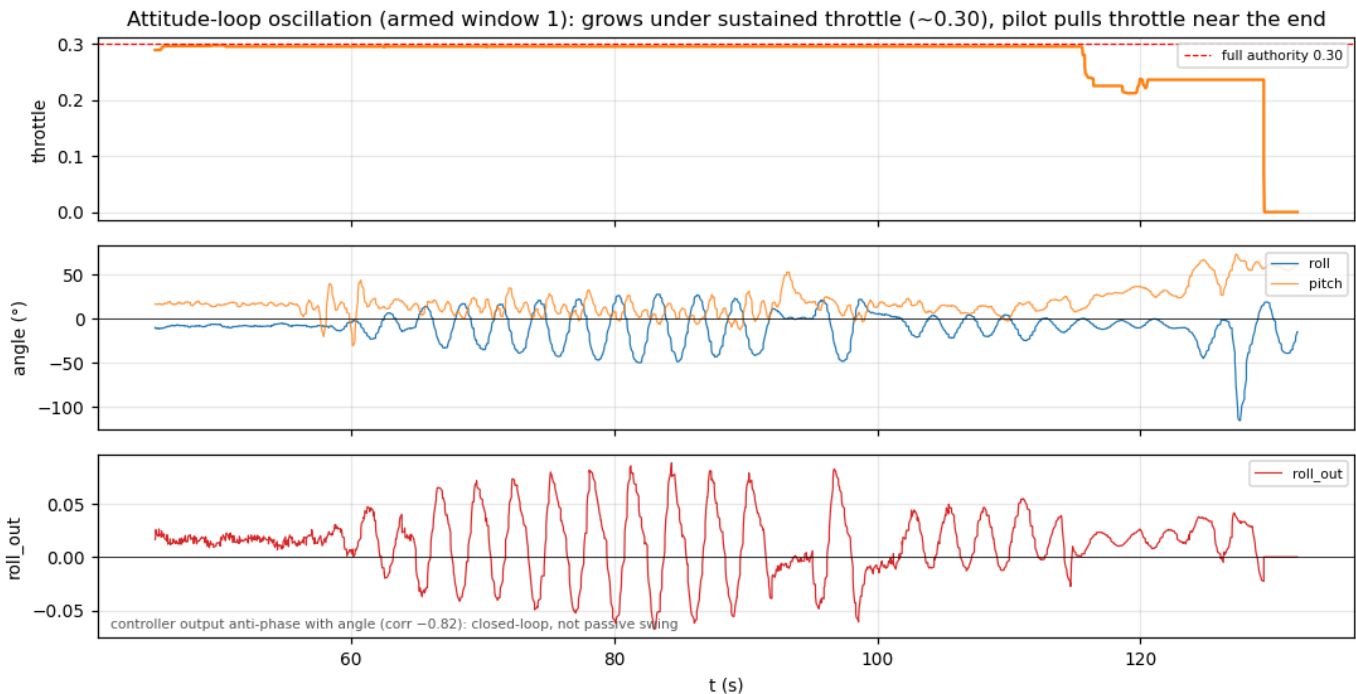


Figure 5: attitude-loop oscillation

Window 1 (above) is the clearest: throttle is held at ~ 0.30 from $t \approx 48$ s; roll starts near $\pm 5^\circ$ and **grows to ± 40 – 50°** (a $\sim 115^\circ$ excursion by $t \approx 128$ s) at a dominant **~ 0.3 Hz** (≈ 3 s period); pitch carries a faster ~ 0.7 – 1.2 Hz component. The operator then pulls throttle ($t \approx 115$ – 128 s) and the system rings down into the disarm. The same build-up repeats in window 2 (roll envelope grows to ~ 60 – 67° , throttle ~ 0.25 , pulled at $t \approx 276$ s) and window 3 (roll grows $12^\circ \rightarrow 58^\circ$, throttle ~ 0.22 , pulled at $t \approx 344$ s).

It is a closed-loop limit cycle, not passive rig swing:

- The controller output is **anti-phase with the angle** — roll_out vs roll_angle_curr correlate **-0.82** over the build-up — i.e. negative feedback actively driving the motion, and the motors modulate with it (M1 swings 0.00–0.56 in the same window).
- The oscillation **scales with PID authority**: it appears and grows at flight-relevant throttle (≈ 0.22 – 0.30 , where the authority ramp opens up) and decays once throttle drops below it. (A per-window throttle $\leftrightarrow$ amplitude correlation is weak/noisy because the build-up and ring-down lag throttle by tens of seconds — it’s a slow divergence with hysteresis, not an instantaneous gain.)

Likely cause. The roll/pitch rate loop runs **$K_p = 0.0005$ with $K_d = 0$ and an integral-dominant $K_i = 0.01$** (i_max 0.2). With essentially no proportional or derivative (damping) term, the loop leans on the integrator, which lags the error — classic recipe for a low-frequency limit cycle against a pendulum-like plant. As the authority ramp raises the effective loop gain with throttle, the closed loop becomes underdamped and oscillates. The ~ 0.3 Hz period is consistent with an integral-driven oscillation on the rig’s slow pivot dynamics.

Caveats. The rig is a constrained pendulum, so the exact frequency/amplitude won't match free hover — but a controller that *diverges into oscillation under its own feedback* on the bench is a genuine red flag, and the anti-phase output proves it's the loop, not the fixture. Re-tune before flight: add rate **Kd** for damping, raise **Kp** off the integrator, and reduce **Ki/i_max**; then re-test.

Yaw — disturbance rejection, saturation, and the hand-clamp episodes

Yaw was **rate-controlled with $\text{yaw_rate_sp} = 0$ for the entire session** (pilot never commanded yaw). Every bit of yaw motion is therefore a *disturbance* — the rig being spun by hand, which the user confirms (“yaw suddenly starts to rotate, which I sometimes hand-clamped”). yaw_rate_curr ranged **$-80...+68$ °/s**.

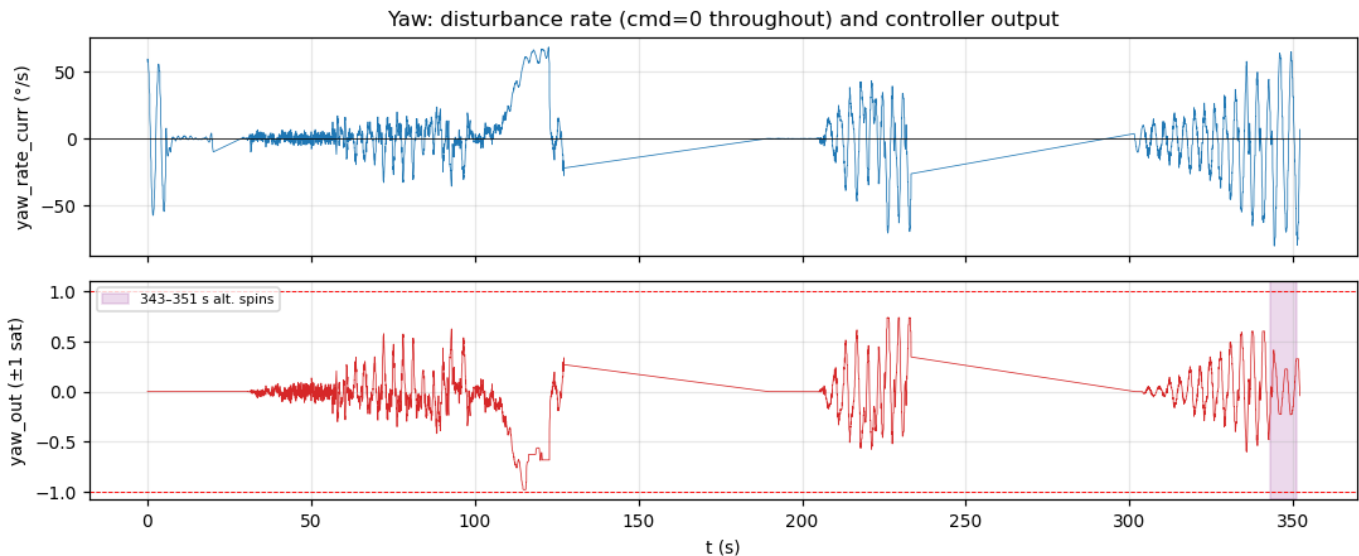


Figure 6: yaw spin episodes

Detected yaw episodes ($|\text{rate}| > 40$ °/s) fall in two regimes:

- **$t = 0-5$ s, $\pm 55-59$ °/s, $\text{yaw_out} = 0.00$.** Free swings with *no* control response — because throttle was near idle, the authority ramp gated the output to zero. The frame spins freely on the rig.
- **$t = 112-123$ s, $216-233$ s, $330-351$ s, up to ± 80 °/s, $\text{yaw_out} \pm 0.6...0.76$.** Here the controller fights hard. yaw_out saturates ($|\cdot| > 0.9$) in 0.5 % of armed frames, and the **343-351 s** stretch is a run of alternating $\pm 65-80$ °/s spins on a ~ 2 s period — consistent with repeated hand-spin $\rightarrow$ clamp cycles (and/or a yaw limit-cycle as the strong yaw loop, $K_p 0.018$, fights the rig).

Because yaw's gain is $36\times$ roll/pitch, yaw is the only axis with meaningful authority in this log — which is exactly why yaw shows large $_{\text{out}}$ while roll/pitch sit near zero. The empirical yaw slope (0.0117) matches $K_p = 0.018$ (reduced by integral/clamp/ramp).

Takeaways

1. **The roll/pitch attitude loop is unstable on the bench** — it diverges into a ~ 0.3 Hz limit cycle (roll to $\pm 50-67^\circ$) whenever throttle is held up, and the operator had to pull throttle to stop it, three times. Output is anti-phase with angle (corr -0.82), so it is the *control loop*, not passive rig swing. **Address this before any flight attempt.**
2. **Likely fix is rate-loop tuning:** $K_p = 5e-4$ with **$K_d = 0$** and integral- dominant $K_i = 0.01$ gives almost no proportional/damping authority — add **Kd** for damping, raise **Kp** off the integrator, lower K_i/i_{max} , re-test on the rig.
3. **Resting pose:** at idle the frame hangs to one side (rig is a free pivot); that tilt is real and not a sensor/fusion error (fusion matches gravity).
4. **Timing is perfect** (4 ms / 1 ms, zero jitter) — the instability is a tuning problem, not a scheduling/loop-rate one.

5. **Yaw loop is ~36× stronger** and saturates against the hand-spin disturbances; the 343–351 s alternating spins may be a separate yaw limit-cycle — re-check after the roll/pitch fix.
6. Next capture is **rig-only**: after re-tuning, prove on the rig that throttle-up no longer oscillates (see [recommendations.md](#) for the protocol). Flight stays off the table until the rig is stable.

Motor analysis — export-20260617-124210.bin

Motor-output behaviour from MotorTelemetry (msgid 1029, 6,562 frames), restricted to **ARMED** windows, cross-referenced with ControlTrace and the firmware mixer. Bench-rig run (never IN_AIR). Companion to [control-loop-analysis.md](#).

Mixer & geometry (from firmware)

Quad-X mixer (src/control/angle_rate_controller.c, include/actuator/actuator.h):

```
out_i = throttle + roll·s_roll[i] + pitch·s_pitch[i] + yaw·s_yaw[i]      (clamped 0..1)
s_roll  = {-1, -1, +1, +1}      s_pitch = {+1, -1, -1, +1}      s_yaw = {+1, -1, +1, -1}
```

MotorTelemetry.cmd[0..3] = M1..M4, normalized **0..1** (idle floor 0.005):

idx	cmd[]	position	spin	yaw sign
0	M1	front-right	CW	+
1	M2	rear-right	CCW	−
2	M3	rear-left	CW	+
3	M4	front-left	CCW	−

Plan view (front at top); each cell = motor · spin · mix signs (roll, pitch, yaw):

	left	right
front	M4 FL · CCW · (+, +, −)	M1 FR · CW · (−, +, +)
rear	M3 RL · CW · (+, −, +)	M2 RR · CCW · (−, −, −)

Signs from firmware $s_{\text{roll}} = \{-1, -1, +1, +1\}$, $s_{\text{pitch}} = \{+1, -1, -1, +1\}$, $s_{\text{yaw}} = \{+1, -1, +1, -1\}$ for M1..M4. CW = M1,M3; CCW = M2,M4.

Output levels — no saturation, low throttle

All ARMED frames:

motor	min	max	mean
M1 FR	0.00	0.56	0.165
M2 RR	0.00	0.62	0.233
M3 RL	0.00	0.69	0.208
M4 FL	0.00	0.59	0.213

Peak 0.69, **0 frames above 0.95** — never saturated, never close. Mean ~0.20 matches the ~0.25 pilot throttle (below the 0.30 full-authority point the whole time, see control-loop doc). This is gentle bench running, not a hover (hover would need all four sustained near ~0.5).

Balance under a level, throttle-only command

The decisive test for “tilted with just throttle”: with roll/pitch commanded ~0 and throttle up, the four motors *should* be near-equal. They aren’t:

motor	mean	vs average (0.247)
M1 FR	0.200	−19 %
M2 RR	0.281	+14 %

motor	mean	vs average (0.247)
M3 RL	0.252	+2 %
M4 FL	0.256	+4 %

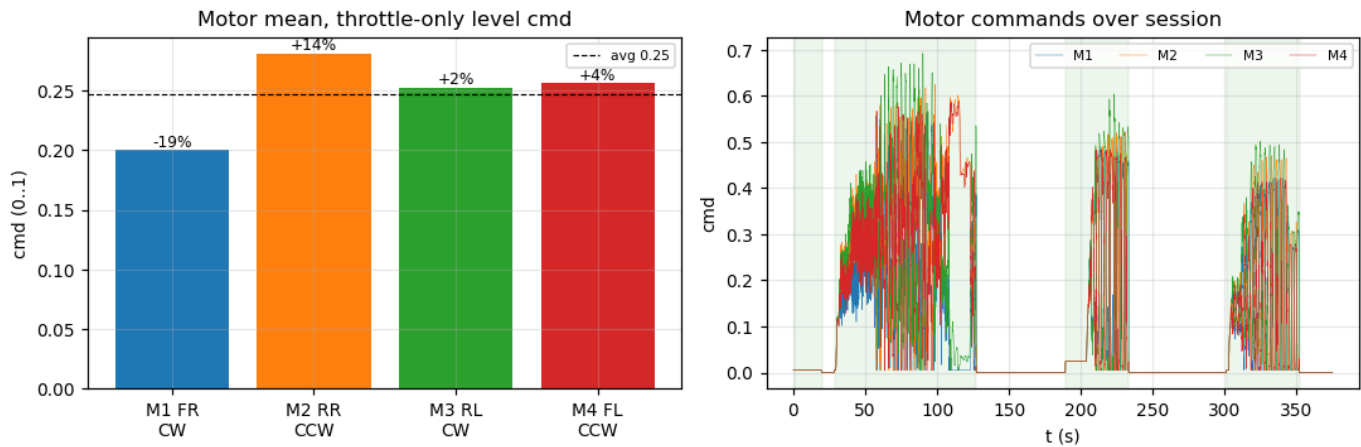


Figure 7: motor balance and timeline

The spread is the **mixer injecting the controller's (small) corrections**, not a raw hardware imbalance — it is fully explained by the logged `roll_out/pitch_out/ yaw_out`:

- **Pitch:** front pair ($M1+M4 = 0.456$) < rear pair ($M2+M3 = 0.533$) by 0.063. Rear thrust > front $\Rightarrow$ nose-down torque — the controller *is* pushing against the $+25^\circ$ nose-up tilt, just weakly (`pitch_out` ≈ -0.023 ; see control-loop doc for why it's so small). Direction correct, magnitude tiny.
- **Yaw:** CW pair ($M1+M3$) – CCW pair ($M2+M4$) = **-0.072**, a persistent yaw differential — the strong yaw loop ($K_p 0.018$) holding against the rig's hand-induced spin.
- **Roll:** right pair slightly reduced vs left, consistent with the -9.7° roll.

So the motor imbalance is a *symptom* of the real tilt + rig yaw disturbance being mixed in, not an independent actuator fault. Because the corrections themselves are small (low roll/pitch authority), the imbalance is small and the frame stays tilted.

What this log can and can't tell us about the motors

- OK Mixer wiring/signs look correct: pitch/roll/yaw corrections map to the right motor groups with the right signs.
- OK No saturation, no zero/stuck motor, no obvious dead actuator (all four span a sensible 0→0.6 range).
- **WARN Cannot assess thrust matching / motor health from a rig run.** Equal-command thrust symmetry needs an *unconstrained* hover (or a thrust-stand per motor). The differences here are control corrections, so a genuinely weak/strong motor would be masked by the loop.
- Note: `MotorTelemetry` carries 8 slots (`cmd[4..7]`); only 0-3 are used (quad).

Recommendations

1. To rule out a real hardware imbalance behind the tilt, do a **per-motor thrust check** (thrust stand or props-off current draw) — the rig log can't separate actuator asymmetry from control action.
2. Don't judge motor balance from this run — once the rig is stable (see [recommendations.md](#)), a per-motor thrust check on a stand is the way to confirm true equal-command balance. (No free flight to assess this.)
3. Mixer signs/geometry verified against firmware — no change needed there.

Kernel / RTOS analysis — export-20260617-124210.bin

Health of the **vaio** RTOS from the PERF telemetry: PerfGlobal (1034, 372), PerfTask (1035, 3,173), PerfFifo (1036, 1,062), and EstPerf (1033, 284). Companion to [session-analysis.md](#).

Headline

The kernel is **healthy and not the bottleneck**: ~63 % CPU load with stable headroom, estimator using 22 % of its loop budget, no heap OOM, no IPC timeouts, all stacks within margin, and — critically — **zero drops on every control-path FIFO**. The only saturation is on the **telemetry** FIFOs and the TX buffer, i.e. the *downlink* can't keep up, which is by-design lossy (overwrite) and does **not** affect flight control.

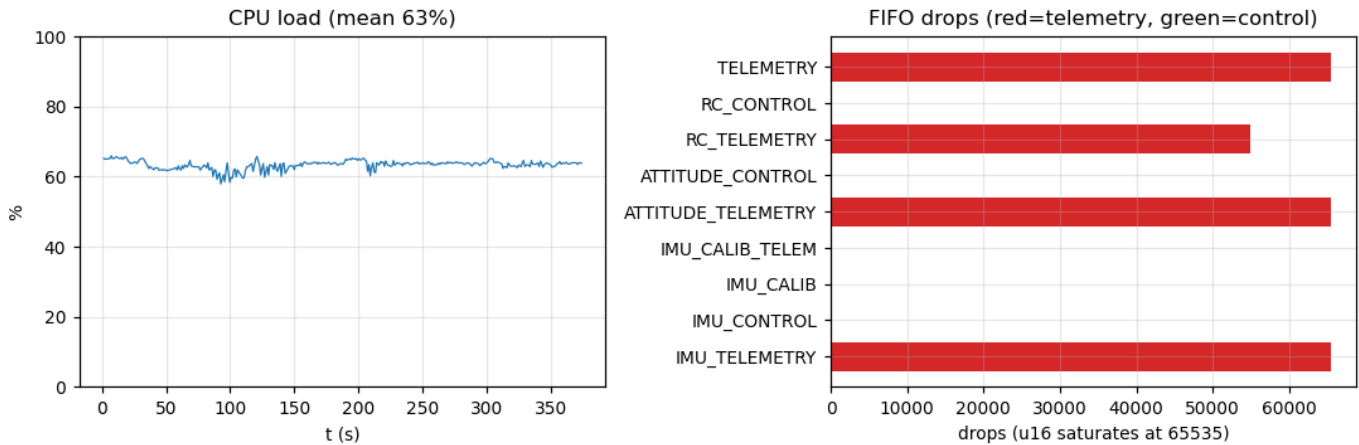
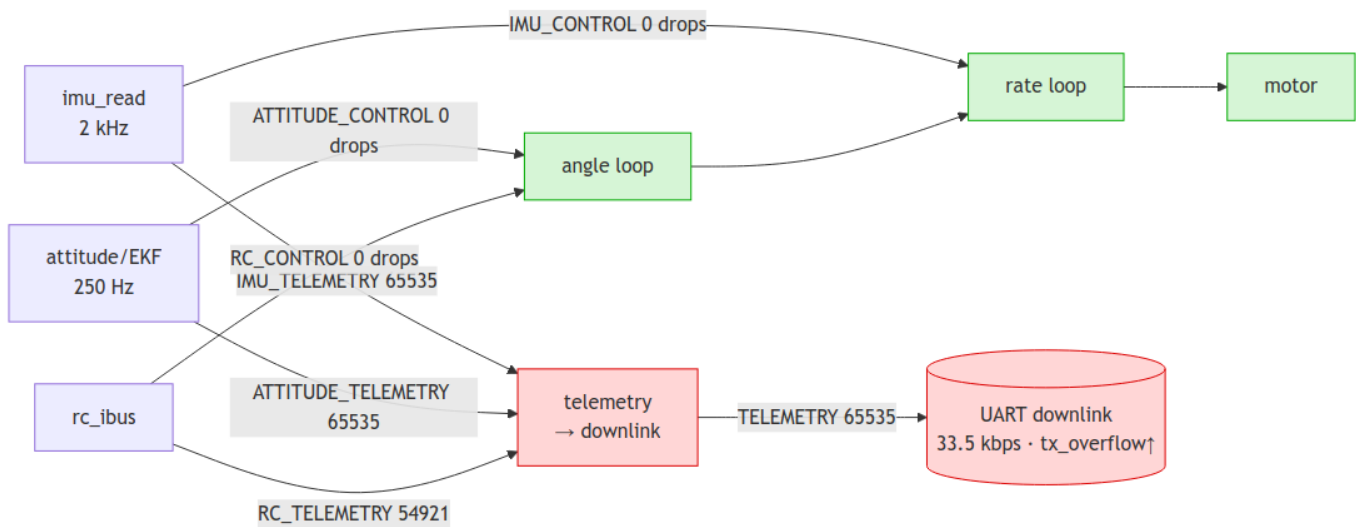


Figure 8: CPU load and FIFO drops

Dataflow: why telemetry drops but control doesn't

SPSC FIFOs connect producers to consumers. The **control** consumers (rate/angle loops) keep up; the **telemetry** consumer (downlink) can't drain at the offered rate, so its FIFOs overwrite. Same producers, two fan-outs, opposite outcomes:



Edge labels: FIFO name + worst-case drops. Control FIFOs = 0; telemetry FIFOs saturate.

CPU & scheduler

- **CPU load** $\approx$ **63 %** (mean 63.3 %, range 57.8–65.9 % over 312 inter-sample deltas). Steady — no creep, no spikes. $\sim$ 37 % idle headroom. > Computed from `cpu_cycles_lo/idle_cycles_lo` **deltas** between consecutive > reports. These are the *low 32 bits* of 64-bit counters, so a single sample's > 1 - idle/cpu is meaningless (gives -239 % etc.); deltas $< \sim 51$ s apart are > valid. **Reporting caveat:** ship the high words (or a precomputed load) so > consumers don't have to reconstruct this.
- **Scheduler:** 6,238,647 context switches over 465 s = **13,414 /s**. `systick_max_cyc` = 1000 (longest tick handler), `systick_preemptions` 348,875.
- **Uptime:** first report at 91 s, last at 465 s — the FC had been running **$\sim$ 91 s before logging started** (explains `tx_overflow` already = 612 at log start).

Estimator timing (EstPerf)

- Runs at a rock-steady **250 Hz**, IMU **decimation 8** (2000 Hz $\rightarrow$ 250 Hz).
- mean **709 μ s**, peak **891 μ s** $\rightarrow$ **18 % mean / 22 % peak** of the 4000 μ s budget. Comfortable. This is the EKF (see [sensor-analysis.md](#) §Fusion).

Memory

- **Heap:** 38 allocs, 2 frees, **0 OOM**, peak **21,232 / 57,344 B (37 %)** — essentially all-at-startup static allocation, no churn, no leak. Healthy.
- **Stacks** (worst-case high-water over the whole session):

task_id	peak / size	%
4	1508 / 2048	74 % $\leftarrow$ highest
13	860 / 2048	42 %
11	788 / 2048	38 %
6	684 / 2048	33 %
5	420 / 1536	27 %
10	252 / 1024	25 %
9	492 / 2048	24 %
1	100 / 512	20 %
8	396 / 2048	19 %
7	156 / 1024	15 %
2 / 12	124 / 1024	12 %

All within margin. **task_id 4 at 74 % is the one to watch** (26 % headroom) — fine for now, but worth keeping an eye on if its workload grows.

Task names were not captured — `PerfTaskname` (1038) requires a `PerfTasknameRequest` (1037) exchange the GCS never sent, so the log carries only numeric ids. To label these, request task names on connect. (Firmware `src/main.c` creates `comm/imu/attitude/rc/angle/rate/motor/telem` perf tasks; mapping ids needs the live exchange to be certain.)

IPC

- `ipc_takes` 3,642,812, `ipc_gives` 3,809,036, `ipc_timeouts` = **0** (clean).
- `ipc_takes_blocked` 2,687,752 (**74 %**) — high but **expected**: worker tasks block waiting on their input FIFOs/semaphores; blocking $\neq$ a problem here given zero timeouts.

FIFOs — the one real signal

SPSC FIFOs, `OVERWRITE` policy (newest wins). Worst-case over the session:

fifo	role	peak fill	drops
IMU_CONTROL (2)	IMU → rate loop	33 %	0
ATTITUDE_CONTROL (6)	attitude → angle loop	22 %	0
RC_CONTROL (8)	RC → angle loop	25 %	0
IMU_TELEMETRY (1)	IMU → downlink	100 %	65535 WARN
ATTITUDE_TELEMETRY (5)	attitude → downlink	100 %	65535 WARN
RC_TELEMETRY (7)	RC → downlink	100 %	54921
TELEMETRY (9)	control → downlink	100 %	65535 WARN
IMU_CALIB (3) / CALIB_TELEM (4)	calibration	0 %	0

The split is the whole story: every **control-path** FIFO never dropped a sample and stayed ≤ 33 % full — the flight loops are fed cleanly. Every **telemetry-path** FIFO is pinned at 100 % and dropping continuously. The downlink simply can't drain them at the offered rate (corroborated by `SystemHealth.tx_overflow 612→2680` and the 33.5 kbps link in the session report). Because the policy is `OVERWRITE`, telemetry always shows the *freshest* sample but loses the ones in between — acceptable for monitoring, and it does **not** starve control.

Reporting caveats: the drop counter is `u16` and **saturated at 65535** — true drop counts are higher and unknowable; widen to `u32` or report a rate. The calibration FIFOs being idle is consistent with no calibration this session.

Takeaways

1. Kernel is healthy: 63 % CPU, 22 % estimator budget, 0 OOM, 0 IPC timeouts, stacks in margin — **not** limiting flight.
2. **Control is never starved** (0 drops on all control FIFOs); the saturation is purely downlink/telemetry — fix at the link/rate level (decimate high-rate streams or apply the proposed stream-rate control packet `0x2007`), not in the kernel.
3. Telemetry-side reporting fixes: send `PerfGlobal` cycle high-words (or a computed CPU load), widen FIFO drops past `u16`, and support a `PerfTaskname` exchange so task ids resolve to names.
4. Watch **task_id 4** stack (74 %).

Sensor & fusion report — export-20260617-124210.bin

Focused look at the **sensors and the attitude fusion**: calibration fidelity, estimator accuracy, and reporting quality. Source is the ImuRaw (full snapshot) and ImuCompressed (delta) streams, AttitudeEuler (fused output), plus SystemHealth/Heartbeat context. All numbers reproducible via `../parse_log.py`. Companion to the [session analysis](#) and [control-loop-analysis.md](#).

Verdict at a glance

Sensor / stage	Calibration / accuracy	Reporting	Verdict
Gyroscope	bias < 0.1 °/s, noise 0.3–0.5 °/s	clean	OK good
Accelerometer	+8 % scale error at rest	clean	WARN needs scale cal
Magnetometer	uncalibrated (hard+soft iron)	reported but unusable	FAIL fail
Temperature	35.6–37.1 °C, sane drift	clean	OK good
Fusion: roll/pitch	matches gravity within –3.8°	body rates = 0 WARN	OK trustworthy
Fusion: yaw/heading	fed by uncalibrated mag	—	FAIL do not trust

No calibration was performed or captured in this session — zero CalibrationStatus messages and nav_state never entered CALIBRATING. The sensors ran on default/whatever-was-stored calibration, which is consistent with the accel/mag errors below. Units in use: **gyro °/s, accel m/s², mag μT, temp °C** (the codec field comment guessing “rad/s” for gyro is wrong — verified against ControlTrace.*_rate_curr, abs max ~100 either way).

Gyroscope — good

Measured over the cleanest stationary window (177–205 s, 36 samples, board still):

axis	bias (°/s)	noise σ (°/s)
x	+0.092	0.348
y	–0.013	0.279
z	+0.033	0.548

Bias is **< 0.1 °/s on every axis** and noise is sub-degree — a well-behaved, well-zeroed gyro. Full-session range is $\pm \sim 100$ °/s during handling, with no clipping or stuck values. This is the one sensor that is clearly trustworthy.

Accelerometer — ~8 % scale error

Gravity magnitude $|acc|$ should read **9.807 m/s²** at rest. It doesn't:

condition	samples	mean $ acc $	error vs g
static, motors OFF	52	10.571	+7.8 %
static, motors ON	32	10.105	+3.0 %

The over-read is **largest with motors off**, so it is **not** vibration rectification (that would push the motors-on figure higher, not lower) — it is a genuine **scale-factor / calibration error of ~8 %**. Per-axis noise at rest is low (σ 0.04–0.08 m/s²), so the sensor is quiet; it's just mis-scaled. A 6-side accelerometer

calibration should bring this in line. Until then, anything that trusts accel magnitude (level estimation, gravity vector, vertical accel) is biased high by $\sim 8\%$.

The motors-on figure being *closer* to g is a coincidence of orientation/linear motion in those samples, not better calibration — treat the **+7.8 % motors-off** number as the real static error.

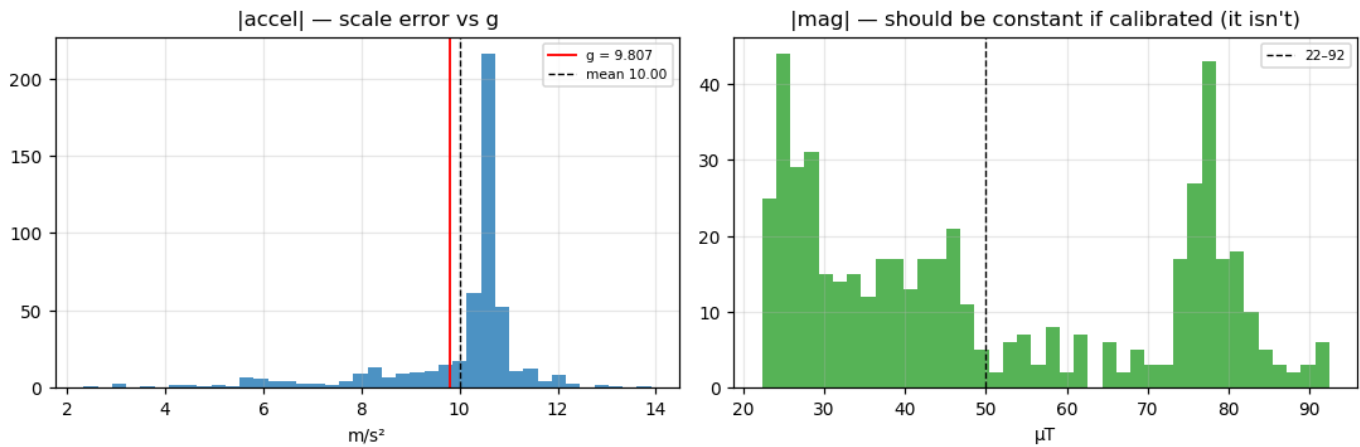


Figure 9: sensor magnitudes

The left panel makes the accel scale error visible (the $|accel|$ mass sits right of the g line); the right panel shows $|mag|$ smeared across 22-92 μT instead of a single calibrated value.

Magnetometer — uncalibrated (do not use as-is)

A calibrated magnetometer reads a near-**constant** field magnitude (Earth's field, $\sim 25\text{--}65\ \mu T$) in every orientation. Here $|mag|$ swings **22.3 $\rightarrow$ 92.4 μT** , a spread of **140 % of the mean** — the signature of an uncorrected hard- and soft-iron field. Per-axis min/max imply large hard-iron offsets:

axis	min	max	hard-iron offset	half-span
x	-87.2	+37.0	-25.1	62.1
y	-37.7	+57.2	+9.7	47.5
z	-29.1	+70.7	+20.8	49.9

Non-zero centers (offsets up to 25 μT) and unequal half-spans (47-62 μT) mean both hard-iron (bias) and soft-iron (scale/skew) correction are missing. The data *is* present and plausibly scaled (μT), but **unusable for heading** until a full ellipsoid-fit magnetometer calibration is run. (Note: this session also ran motors, whose current adds field distortion — calibrate *with the frame powered* and ideally characterize throttle-dependent interference.)

A calibrated mag scatters as a sphere centered on the origin. These projections are off-center, squashed clouds — the visual signature of hard+soft iron:

Temperature — good

IMU temp reads **35.6 - 37.1 $^{\circ}C$** with a gentle warm-up drift (35.9 $\rightarrow$ 36.4 $^{\circ}C$ over the session). Sensible absolute values and smooth — no glitches. (No evidence of temperature compensation being applied to gyro/accel, but the channel itself reports correctly.)

Sensor fusion (attitude estimation)

The fused attitude (AttitudeEuler, msgid 1026) is produced by an **error-state EKF** (6-state: attitude error + gyro bias) running at **250 Hz** (IMU decimation 8 from 2000 Hz). Accel pins **roll/pitch** via the gravity

Magnetometer scatter — off-center, non-spherical = uncalibrated

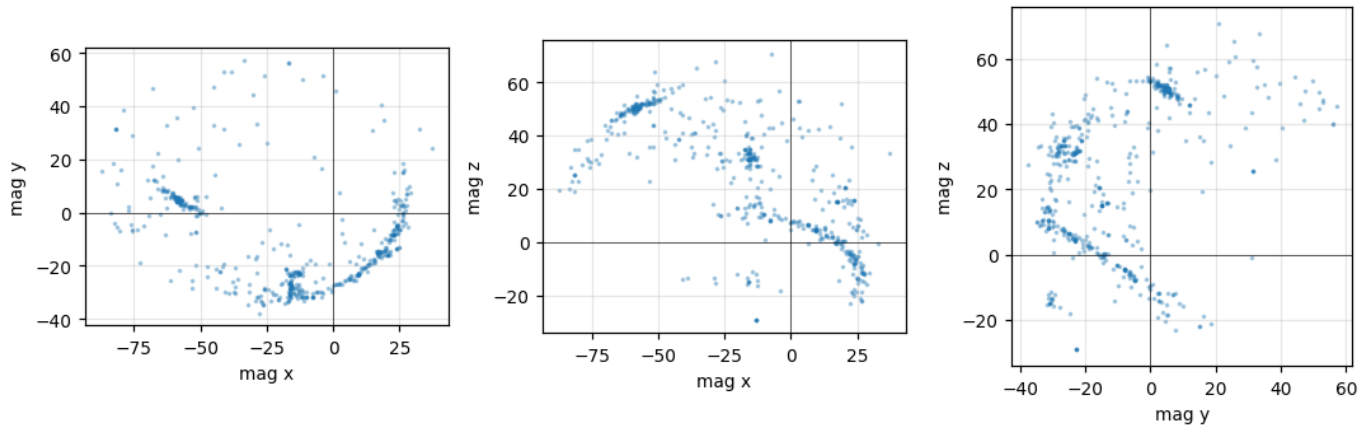
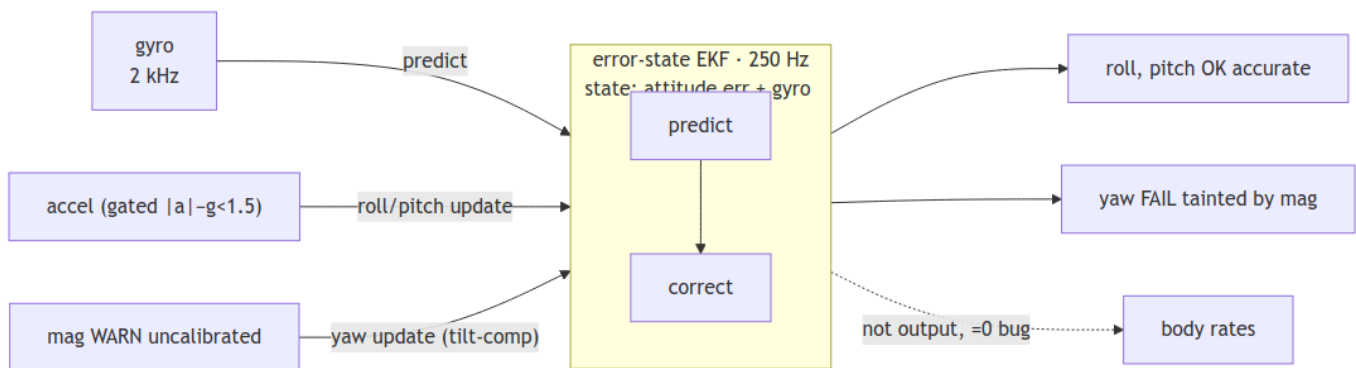


Figure 10: magnetometer scatter

direction (gated when $|a| - g| > 1.5 \text{ m/s}^2$ to reject dynamic accel); the magnetometer pins **yaw only** (tilt-compensated); gyro bias is estimated online. Output is degrees, NED, with a body→world quaternion (firmware `src/est/ekf.c`, `src/est/attitude_task.c`).

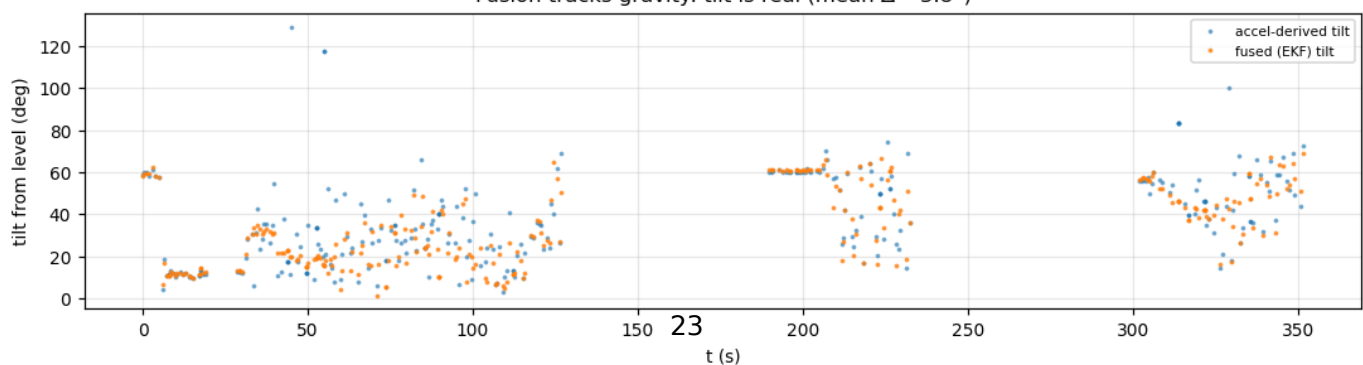


Roll/pitch fusion is accurate — the bench tilt is real

The key cross-check for the “tilted with just throttle” observation: does the estimator actually track gravity, or is it inventing a tilt? Comparing the **fused total tilt** to the **accelerometer-derived tilt** (angle of the accel vector from the at-rest gravity direction, derived from frames the EKF believed level) over all ARMED IMU samples:

	mean	std
accel-derived tilt	37.5°	21.3°
fused tilt	33.7°	18.3°
fused – accel	−3.8°	14.7°

Fusion tracks gravity: tilt is real (mean $\Delta = -3.8^\circ$)



Yaw/heading is not trustworthy

Yaw is pinned by the magnetometer, and **the magnetometer is uncalibrated** ($|mag|$ swings 22–92 μT , hard-iron offsets to 25 μT — see §Magnetometer). An uncorrected mag drags the yaw update toward a distorted heading, so **the fused yaw / heading in this log should not be relied on**. Roll/pitch are unaffected (they come from accel + gyro, not mag). This matters less in ANGLE/ACRO flight (yaw is rate-controlled off the gyro), but any heading-hold or nav use needs the mag calibrated first.

Estimator health & the +8 % accel scale

- The estimator never forced a failsafe: all FAILSAFE transitions correlate with **RC loss**, not estimator degraded (see session report) — so the EKF stayed healthy throughout despite the aggressive rig handling.
- The **+8 % accel scale error** biases the gravity-vector magnitude but not its *direction*, so roll/pitch are largely unaffected; however it eats into the $|a|-g| > 1.5 \text{ m/s}^2$ accept gate (at rest $|a| \approx 10.6 \Rightarrow |a|-g \approx 0.8$, already over half the gate), so once calibrated the estimator will accept *more* static samples and lean less on gyro coasting. Worth fixing for fusion robustness, not just for the raw channel.
- Online gyro-bias estimation has little to do here — the raw gyro bias is already $< 0.1 \text{ }^\circ/\text{s}$.

Reporting fidelity

The IMU uses a **keyframe + delta** scheme:

stream	rate	payload	role
ImuRaw	~1.6 Hz (median dt 621 ms, max gap 2.44 s)	10× f32 (acc,gyr,mag,temp)	absolute keyframe
ImuCompressed	~38 Hz (median dt 26 ms)	10× f16 delta	high-rate deltas

8,764 delta frames vs 508 keyframes (**17.3×**). The delta half-floats decode to real, non-trivial values, so the compression path is alive. **But two reporting bugs undercut it:**

1. **ImuCompressed.ref_seq is stuck at 0** for all 8,764 frames (only one distinct value). The whole point of ref_seq is to bind each delta to the keyframe it was differenced against; with it pinned to 0, a consumer **cannot reliably reconstruct absolute IMU** from the compressed stream — it can't tell which of the 508 keyframes a delta belongs to. Combined with keyframe gaps up to 2.44 s, blind delta accumulation will drift. **This is a fidelity bug, not just cosmetic.**
2. **ImuRaw.sample_time_us is always 0**. No per-sample hardware timestamp, so IMU frames can only be timed by their telemetry-arrival t_{us} (jittery, subject to the link stalls noted in the session report). Real sample timing is lost.

Also relevant to “sensor reporting”: AttitudeEuler **body rates (rollspeed/pitchspeed/yawspeed) are always 0** — the fused-attitude angular rates aren't populated, even though the raw gyro carries them (see the [session report](#), issue #1).

Recommendations (sensor priority order)

1. **Run accelerometer calibration** (6-side) — remove the ~8 % scale error before trusting any accel-derived estimate.
2. **Run magnetometer calibration** (ellipsoid fit, frame powered) — current mag is unusable for heading; characterize motor-current interference while at it. This is also a **fusion** fix: the EKF pins yaw from the mag, so fused heading is untrustworthy until the mag is calibrated (roll/pitch are unaffected).
3. **Fix ImuCompressed.ref_seq** to carry the real keyframe seq so the delta stream is reconstructable; consider a faster ImuRaw keyframe cadence (the 2.4 s gaps are large for a 38 Hz delta stream).
4. **Populate ImuRaw.sample_time_us** so IMU samples can be time-aligned from their own payload.

5. **Capture a calibration session** (drive CALIBRATING + CalibrationStatus) and re-log so the stored cal can be verified against fresh data.
6. Gyro and temperature need no action — they're reporting correctly.

Recommendations & next-run plan — export-20260617-124210.bin

What to fix before the next trial, and exactly what to capture when you re-run. Derived from the analysis in this folder ([session](#) · [control](#) · [motors](#) · [kernel](#) · [sensors+fusion](#)).

Scope: stay on the rig. No free flight yet. The goal of the next run is to get a **stable, well-instrumented rig session** — fix the oscillation and prove it on the bench first. Everything below is rig-only.

1. Fixes before the next rig trial (priority order)

1a. Blocker — the attitude-loop oscillation

This is the one that must change before anything else; right now the roll/pitch loop diverges into a limit cycle whenever throttle is held up.

- **Re-tune the roll/pitch rate loop for damping.** The current tuning is the likely cause: almost no proportional term and **no derivative ($K_d = 0$)**, leaning on a slow integrator. Add some rate **Kd** (damping), raise **Kp** so it isn't integral-dominated, and reduce **Ki** / the integral clamp.
- **Change one thing at a time** and re-test on the rig after each step — small, conservative moves, not a big jump.
- **Tune one axis first** (roll or pitch, whichever the rig lets you isolate), confirm it's stable, then mirror to the other.
- Treat **yaw separately** — it's a different (much stronger) loop and showed its own possible limit-cycle; only revisit it after roll/pitch is calm.

1b. Sensor calibration (do before tuning, so the loop sees good data)

- **Accelerometer 6-side calibration** — removes the ~8 % scale error.
- **Magnetometer calibration with the frame powered** (motors drawing current) — the mag is uncalibrated and currently makes the fused *yaw/heading* untrustworthy.
- **Re-verify after:** at rest `|accel|` should read ~g, and `|mag|` should be roughly constant as you rotate the frame.

1c. Telemetry fields — so the *next* log is fully analysable

These are what made parts of *this* log hard to read. Fix them and the next capture tells you far more:

- **Populate the fused body rates** (attitude angular-rate fields are all zero today) — without them you can't see the rate loop directly; it's the single most useful fix for tuning analysis.
- **Populate the per-sample IMU timestamp** (zero today) so IMU can be time-aligned.
- **Fix the compressed-IMU keyframe reference** (stuck at zero) so the high-rate IMU stream can be reconstructed.
- **Populate or drop CPU-load** (zero today; real timing lives in the perf stream).
- **Resolve the degrees-vs-radians inconsistency** between the control trace and the attitude message (or document per-field units).

1d. Telemetry link

- The downlink was **saturated** (continuous TX-buffer overflow, telemetry FIFOs 100 % full). For a focused tuning run, **prioritise the streams you need** (control trace, attitude, IMU) and trim the rest, or raise the link bitrate.
- **Widen the FIFO drop counter** (it saturates at 65535, so true drops are unknown).

2. What to capture in the next rig run (test protocol)

Run these as distinct, labelled phases so each is easy to find in the log. Keep a **hard throttle cap** and pull throttle the moment oscillation grows.

1. **Calibration pass** — run accel/gyro/mag calibration and capture it (the calibration-status messages + the CALIBRATING state). Confirm it stores.
2. **Static baseline** — armed, motors at idle, frame held at its level reference on the rig for ~15–20 s. Confirms post-cal sensor health and the resting pose.
3. **Throttle-step sweep (the key test)**. With sticks centred, raise throttle in small steps (e.g. ~0.15 → 0.20 → 0.25 → 0.30 → 0.35), **holding each step ~10 s**. Watch for the oscillation-onset throttle. After re-tuning, the goal is **no growing oscillation at any step** — this is how you prove the fix.
4. **Disturbance-and-release**. At a safe, steady throttle, nudge the frame a few degrees by hand and let go; capture the recovery. A good tune **settles with decaying overshoot**; a bad one keeps ringing. Repeat on roll and pitch.
5. **Yaw check**. Hold yaw, then give one controlled yaw nudge and release — confirm it returns without a sustained spin/limit-cycle.

Keep the **same rig setup** between runs (which axis is free, CG, props on/off) so results are comparable, and note it.

3. What specifically to capture (states & methods)

So each capture is self-documenting and the oscillation is well-sampled:

- **Record the active PID gains in the log itself** (read them back over the link at the start of the run). Then every capture says *which tuning* it tested — no guessing later.
- **Mark the phases**. Drop a marker at each phase boundary — a status-text line, or a dedicated RC-switch toggle, or just written-down timestamps — so phases 1–5 above are trivially segmentable.
- **Log the state machine and command acknowledgements** (`nav_state` transitions, arm/cal/set-gain ACKs) so you can confirm what was actually applied.
- **Raise the rates of the signals that matter** for the duration of the test: the control trace, attitude (with the now-populated body rates), and full IMU. The oscillation is slow (~0.3–1.2 Hz), so even modest rates resolve it — but the IMU keyframe was very sparse (~1.6 Hz) last time; bump it.
- **Keep time-sync running** so timestamps line up across streams.
- **Note rig/environment** out-of-band: tether, CG, whether props are on, supply voltage.

4. Rig acceptance criteria (gate before *any* flight thinking)

Consider the rig “passed” only when, in one capture:

- ☐ Throttle held through the full sweep (incl. $\geq$ the full-authority point) for $\geq \sim 30$ s with **no growing oscillation**.
- ☐ Disturbance-release **settles** (decaying, no sustained limit cycle) on roll and pitch, and yaw returns cleanly.
- ☐ Sensors calibrated: $|accel| \approx g$ at rest, $|mag| \sim \text{constant}$ when rotated.
- ☐ Telemetry fields populated (body rates, IMU timestamp) and link not saturated.

Only after all four — and reviewing that capture — move to a **tethered / very low** check. **Open free flight stays off the table until the rig is boring.**